The Journal of Open Source Software

IoT Kubernetes MQTT Network Simulator: A Cloud-Native Platform for Realistic IoT Device and Network Behavior Simulation

Raj Singh Gaur [1]

1 Independent Researcher

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor:

Submitted: 20 June 2025
Published: unpublished

License
Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

DRAFT

Summary

The Internet of Things (IoT) ecosystem requires robust testing and simulation platforms to evaluate device behavior, network protocols, and system resilience at scale. Existing IoT simulators often lack modern cloud-native deployment capabilities, realistic failure modeling, or easy reproducibility. We present an open-source IoT network simulator that leverages Kubernetes orchestration and MQTT messaging to provide a realistic, scalable, and easily deployable simulation environment.

The simulator deploys IoT devices as containerized Kubernetes pods, each running a Python application that communicates via MQTT through a shared broker. It supports configurable network conditions (latency, packet loss), device failures, and resource constraints that mirror real-world IoT deployments. The platform enables researchers and educators to conduct reproducible experiments on IoT network behavior, protocol performance, and system resilience without requiring physical hardware.

Statement of Need

Modern IoT research and development requires simulation platforms that can: 1. **Scale efficiently** from dozens to hundreds of devices 2. **Model realistic network conditions** including latency, packet loss, and failures
3. **Deploy easily** across different computing environments 4. **Ensure reproducibility** of experimental results 5. **Support modern protocols** like MQTT with proper failure semantics

Existing simulators like Cooja, NS-3, and iFogSim address some of these needs but lack cloud-native deployment, comprehensive failure modeling, or easy setup for educational use. This simulator fills the gap by providing a Kubernetes-native platform that combines ease of deployment with realistic IoT behavior modeling.

Key Features

- **Cloud-native deployment** using Kubernetes for easy scaling and orchestration
- **Realistic MQTT simulation** with proper client inactivity and broker timeout modeling
- **Configurable network conditions** including latency (10-500ms) and packet loss (0-20%)
- **Device failure simulation** with configurable failure probability and recovery time
- **Resource constraints** that mirror real IoT device limitations (CPU/memory limits)
- **Comprehensive metrics collection** including message counts, latency, and failure statistics
- **Automated experiment workflows** with log collection and result processing
- **Full reproducibility** with containerized deployment and version-controlled configurations

Implementation

The simulator consists of several key components:

1. **IoT Device Simulator** (`mqtt_device.py`): A Python Flask application that simulates IoT device behavior, including MQTT communication, periodic heartbeats, and configurable failure scenarios.
2. **Kubernetes Orchestration**: YAML configurations for deploying device pods, MQTT broker, and supporting infrastructure with parameterized scaling and resource allocation.
3. **Network Simulation**: Software-based simulation of network latency, packet loss, and device connectivity issues within each device pod.
4. **Experiment Automation**: Shell scripts for running timed experiments, collecting logs, and processing results across multiple parameter configurations.
5. **MQTT Broker**: Containerized Eclipse Mosquitto deployment with appropriate configuration for IoT device simulation.

The system has been validated through comprehensive experiments testing scalability (5-50 devices), network conditions (varying latency/loss), failure scenarios (different failure rates/durations), and resource constraints (CPU/memory limits).

Usage Example

```
# Clone and setup
git clone https://github.com/RajSinghGaur/IoT-Kubernetes-MQTT-Network-Simulator
cd IoT-Kubernetes-MQTT-Network-Simulator

# Configure Docker Hub username
echo "DOCKERHUB_USERNAME=your-username" > .env

# Build and deploy
docker build -t your-username/iot-mqtt:latest .
docker push your-username/iot-mqtt:latest
kubectl apply -f mqtt-broker.yaml
envsubst < k8s.yaml.template > k8s.yaml
kubectl apply -f k8s.yaml

# Run automated experiment
./run_experiment.sh 1200  # 20-minute experiment
```

Educational and Research Applications

This simulator serves multiple use cases:

- **IoT Protocol Research**: Testing MQTT performance, reliability, and scaling characteristics
- **Network Behavior Studies**: Analyzing the impact of network conditions on IoT communication
- **Failure Recovery Analysis**: Studying device failure patterns and recovery mechanisms
- **Educational Tool**: Teaching Kubernetes, MQTT, and IoT concepts with hands-on experiments
- **System Prototyping**: Testing IoT applications before physical deployment

DRAFT

Validation and Results

The simulator has been validated through extensive experiments demonstrating: - Linear scaling behavior up to 50 devices with predictable latency increases - Realistic response to network condition changes (latency/loss simulation) - Proper failure/recovery behavior matching real MQTT client semantics - Resource constraint handling typical of IoT device limitations

Results show average message latencies of 1-3ms with good scalability characteristics and proper failure modeling that matches real-world IoT deployment behavior.

Conclusion

The IoT Kubernetes MQTT Network Simulator provides a modern, cloud-native platform for IoT simulation that combines ease of deployment with realistic behavior modeling. Its open-source nature, comprehensive documentation, and reproducible experimental framework make it valuable for both research and educational applications. The simulator fills an important gap in available IoT simulation tools by providing Kubernetes-native deployment with proper MQTT protocol semantics and failure modeling.

Acknowledgements

We acknowledge the open-source community for the foundational tools that made this work possible, including Kubernetes, Eclipse Mosquitto, and the Python MQTT client library.

References

DRAFT